OTEL-07: Dynatrace OTLP Integration

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 7 of 8 | **Created:** January 2026 | **Last Updated:** 10/28/2026

Complete Setup for OpenTelemetry with Dynatrace

This notebook provides end-to-end configuration for sending OpenTelemetry data to Dynatrace, including authentication, endpoints, and verification.

Table of Contents

1. Dynatrace OTLP Endpoints
2. Authentication Setup
3. Collector Configuration
4. Direct SDK Export
5. Entity Mapping
6. Span Attributes for Dynatrace
7. OTLP Metric Dimensions
8. Verification
9. Coexistence with OneAgent and DynaKube

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with OTLP enabled

Requirement	Details
Permissions	Token creation access
Knowledge	OTEL-01 through OTEL-06

SaaS 1.337: OpenTelemetry-relevant changes

1. **OTel `service.name` in SDv1 service names.** Per the release notes, *"You can now use the OTel service.name attribute to customize the name of SDv1 services."* Smartscape shows `service.name (detected name)`, and the change *"affects the service name in all Smartscape use cases, and the dt.service.name attribute on spans and metrics."* `dt.service.name` is on every span — OneAgent and OTLP alike — so it is the field to group by:
   `dql fetch spans, from:-1h | filter isNotNull(dt.service.name) | summarize span_count = count(), by:{dt.service.name, dt.smartscape.service}`
2. **OneAgent + OpenTelemetry-injector coexistence matrix** — documented in **K8S-11 § 2a**. Covers double-instrumentation symptoms, init-container ordering, `LD_PRELOAD` / `JAVA_TOOL_OPTIONS` clobber, trace-context conflicts, and a per-namespace decision flow. Read it before planning OTel auto-instrumentation where OneAgent is also deployed.
3. **SDv2 for AWS Lambda (Early Access).** *"It provides a single unified ruleset for both OpenTelemetry and OneAgent instrumentation."* See **CLOUD-04 § Sprint 1.337**.

> **Sources:** SaaS 1.337 release notes (DT docs).

---

1. Dynatrace OTLP Endpoints

SaaS Endpoints

Dynatrace supports **OTLP/HTTP only** for native ingest, with binary Protocol Buffers — *"gRPC is not supported. API calls need to use HTTP."* and *"JSON is not supported for Protocol Buffers. Binary format must be used."* Use a Collector to convert gRPC to HTTP.

Signal	HTTP Endpoint
All (base)	`https://{env-id}.live.dynatrace.com/api/v2/otlp`
Traces	`https://{env-id}.live.dynatrace.com/api/v2/otlp/v1/traces`
Metrics	`https://{env-id}.live.dynatrace.com/api/v2/otlp/v1/metrics`
Logs	`https://{env-id}.live.dynatrace.com/api/v2/otlp/v1/logs`

> **Important:** gRPC is **not supported** for direct Dynatrace ingest. If your SDKs use gRPC, route through a Collector with an `otlp` gRPC receiver and `otlphttp` exporter. See Transform OTLP gRPC.

ActiveGate Endpoints

For on-premises or network-restricted environments:

Signal	Endpoint
All	`https://{activegate-host}:9999/e/{env-id}/api/v2/otlp`

Dynatrace Collector Distribution

Dynatrace provides its own **Collector distribution** with verified, production-ready components:

Aspect	Detail
Image	`ghcr.io/dynatrace/dynatrace-otel-collector/dynatrace-otel-collector`
Features	Pre-configured for Dynatrace, upstream-compatible components
Docs	Dynatrace Collector

> **Tip:** The Dynatrace Collector distribution is recommended for production deployments. It includes verified components and stays current with upstream releases.

Environment ID

Find your environment ID:
1. Log into Dynatrace
2. Look at the URL: `https://abc12345.live.dynatrace.com`
3. `abc12345` is your environment ID

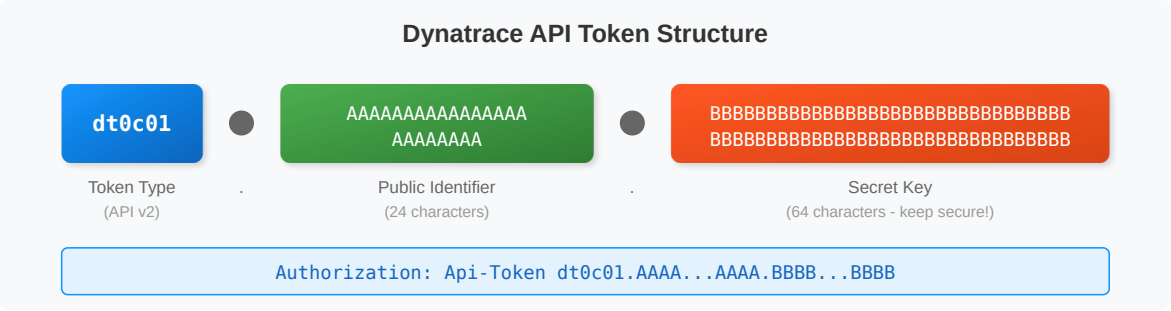
2. Authentication Setup

Create API Token

1. Navigate to **Settings > Access tokens**
2. Click **Generate new token**
3. Name: `otel-ingest-token`
4. Select scopes:

Scope	Signal	Required
`openTelemetryTrace.ingest`	Traces	Yes (for traces)
`metrics.ingest`	Metrics	Yes (for metrics)
`logs.ingest`	Logs	Yes (for logs)

Token Format



Header Format

Use the `Api-Token` prefix in the Authorization header:

```
Authorization: Api-Token &lt;your-full-token&gt;
```

Token Type and Auth Scheme

The `Api-Token` scheme above is for **classic API Tokens** (prefix `dt0c01.*`). **Platform Tokens** (prefix `dt0u01.*` for user tokens, `dt0s16.*` for service-user tokens) use `Bearer ${TOKEN}` instead:

```
# Classic API Token
Authorization: Api-Token dt0c01.XXXXXXXX...

# Platform Token (Gen3-preferred)
Authorization: Bearer dt0u01.XXXXXXXX...
```

Using the wrong scheme returns **401 Unauthorized** even when the token has the correct scopes — a frequent first-deployment failure mode. Platform Tokens are preferred in Gen3 environments because they support fine-grained scopes via IAM policies. For OTLP ingest both token types work, with different scopes: a Platform Token needs `openpipeline:traces:ingest`, `openpipeline:metrics:ingest` and `openpipeline:logs:ingest`; a classic token needs the scopes in the table above.

> **Sources:** OTLP API (DT docs) — *"To authenticate, use a platform token with `Bearer` in the `Authorization` HTTP header." "The required scopes are `openpipeline:logs:ingest`, `openpipeline:metrics:ingest`, and `openpipeline:traces:ingest`."*; API authentication (DT docs) — `openTelemetryTrace.ingest`: *"Grants permission to ingest OpenTelemetry traces."*

3. Collector Configuration

Image Versioning Best Practice

> **Important:** Always pin your OTel Collector image to a specific version. Using `latest` can cause unexpected behavior during upgrades.

```
# Avoid - Non-deterministic deployments
image: otel/opentelemetry-collector-contrib:latest

# Recommended - Pin to a specific version
image: otel/opentelemetry-collector-contrib:0.151.0

# Or use the Dynatrace distribution (pinned, not :latest)
image: ghcr.io/dynatrace/dynatrace-otel-collector/dynatrace-otel-
collector:v0.48.0
```

Check the OpenTelemetry Collector releases or Dynatrace Collector releases for the current stable version.

Complete Collector Config for Dynatrace

```
# otel-collector-config.yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 10s
    send_batch_size: 1000

  memory_limiter:
    check_interval: 1s
    limit_mib: 800
    spike_limit_mib: 200

  # Add service.name if missing
  resource:
    attributes:
      - key: service.name
        value: "unknown-service"
        action: insert

exporters:
  otlphttp/dynatrace:
    endpoint: https://${DT_ENV_ID}.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: Api-Token ${DT_API_TOKEN}
    retry_on_failure:
      enabled: true
      initial_interval: 5s
      max_interval: 30s
      max_elapsed_time: 300s

  debug:
    verbosity: detailed

extensions:
  health_check:
    endpoint: 0.0.0.0:13133

service:
```

```
  extensions: [health_check]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [otlphttp/dynatrace]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [otlphttp/dynatrace]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, resource, batch]
      exporters: [otlphttp/dynatrace]
```

Resource Sizing Guide

Environment	CPU Limit	Memory Limit	Batch Size
Dev/Test	200m	256Mi	500
Staging	500m	512Mi	1000
Production	1000m	1Gi	2000

Running with Environment Variables

```
export DT_ENV_ID="abc12345"
export DT_API_TOKEN="&lt;your-api-token&gt;"

otelcol-contrib --config otel-collector-config.yaml
```

4. Direct SDK Export

Python Direct to Dynatrace

```
import os
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import
OTLPSpanExporter
from opentelemetry.sdk.resources import Resource

# Configuration
DT_ENV_ID = os.environ["DT_ENV_ID"]
DT_API_TOKEN = os.environ["DT_API_TOKEN"]

# Resource with service name
resource = Resource.create({
    "service.name": "my-python-app",
    "service.version": "1.0.0",
    "deployment.environment.name": "production"
})

# Setup tracer
provider = TracerProvider(resource=resource)
exporter = OTLPSpanExporter(
    endpoint=f"https://{DT_ENV_ID}.live.dynatrace.com/api/v2/otlp/v1/traces",
    headers={"Authorization": f"Api-Token {DT_API_TOKEN}"}
)
provider.add_span_processor(BatchSpanProcessor(exporter))
trace.set_tracer_provider(provider)
```

Java with System Properties

```
# Set environment variables first
export DT_ENV_ID="abc12345"
export DT_API_TOKEN="&lt;your-api-token&gt;"

# URL-encode the token for the header (space becomes %20)
java -javaagent:opentelemetry-javaagent.jar \
  -Dotel.service.name=my-java-app \
  -
Dotel.exporter.otlp.endpoint=https://${DT_ENV_ID}.live.dynatrace.com/api/v2/ot
lp \
  -Dotel.exporter.otlp.headers="Authorization=Api-Token%20${DT_API_TOKEN}" \
  -jar app.jar
```

Node.js

```
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-
http');
const { resourceFromAttributes } = require('@opentelemetry/resources');  // JS
SDK 2.x+: the Resource class is no longer exported

const sdk = new NodeSDK({
  resource: resourceFromAttributes({
    'service.name': 'my-node-app',
  }),
  traceExporter: new OTLPTraceExporter({
    url:
`https://${process.env.DT_ENV_ID}.live.dynatrace.com/api/v2/otlp/v1/traces`,
    headers: {
      'Authorization': `Api-Token ${process.env.DT_API_TOKEN}`,
    },
  }),
});

sdk.start();
```

> **Sources:** Upgrade to OpenTelemetry JS SDK 2.x (OpenTelemetry GitHub) — *"The `Resource` class is no longer exported; instead use new utility functions."*

5. Entity Mapping

How Dynatrace Maps OTel Data

OTel Resource Attribute	Dynatrace Entity	Notes
`service.name`	Service	Names the service; without it OTel SDKs report `unknown_service`
`service.namespace`	—	Scopes `service.name`; two services may share a name in different namespaces
`host.name`	Host	Links to host entity
`k8s.namespace.name`	K8s namespace	Links to K8s entities
`k8s.pod.name`	K8s pod	Links to the pod

For the exact attributes Dynatrace uses to map OTLP data to entities, see the enrichment and resource-attribute guidance on the OTLP API (DT docs) page — *"make sure your traces have the correct mapping resource attributes set."*

Essential Resource Attributes

Always set these for proper entity mapping:

```
resource = Resource.create({
    # Required
    "service.name": "checkout-api",

    # Recommended
    "service.version": "1.2.3",
    "service.namespace": "ecommerce",
    "deployment.environment.name": "production",

    # For K8s
    "k8s.namespace.name": "checkout",
    "k8s.pod.name": "checkout-api-abc123",
    "k8s.deployment.name": "checkout-api",
})
```

6. Span Attributes for Dynatrace

Dynatrace fields you should not set yourself

Fields in the `dt.*` namespace are populated by Dynatrace. `dt.span.type` and `dt.source`, which older material lists as attributes to set, have no row in the semantic dictionary, and `dt.entity.*` fields are `deprecated`. Send standard semantic-convention attributes and let Dynatrace derive its own fields.

> **Dictionary:** no row for `dt.span.type` or `dt.source`, read 10/02/2026 (control: `dt.openpipeline.source` returned a row).

Semantic Conventions Dynatrace Uses

Convention	Dynatrace Feature
`http.*`	HTTP service detection
`db.*`	Database call analysis
`messaging.*`	Message queue tracking
`rpc.*`	RPC call tracking

Example with Rich Attributes

```
with tracer.start_as_current_span("checkout") as span:
    # Standard semantic conventions
    span.set_attribute("http.request.method", "POST")
    span.set_attribute("url.full", "https://api.example.com/checkout")
    span.set_attribute("http.response.status_code", 200)
    span.set_attribute("http.route", "/checkout")

    # Business context
    span.set_attribute("order.id", order_id)
    span.set_attribute("order.total", order_total)
```

Complex attribute types (SaaS 1.344)

SaaS 1.344+ (staged tenant rollout from 07/29/2026). Per the release notes, *"Dynatrace now preserves complex attribute values on ingested OTLP spans. Nested key-value list attributes (maps) are now supported, keeping their structure intact."* (SaaS 1.344 release notes (DT docs))

Verify the release has reached your tenant before you start exporting nested attributes. How a pre-1.344 tenant treats a nested attribute is not something to discover from production trace data; send one span from a test service and confirm the structure arrives intact.

Flat scalar attributes are not superseded. They remain the right default for anything you filter, group, or alert on — they are cheaper to query and they keep cardinality legible (see **OTEL-04 § 8 — Attribute Guidelines**). A nested structure is opaque to a `by:{...}` clause in the way a flat key is not.

Attribute shape	Use when	Example
Flat scalar	The value is a query dimension — you filter, group, sort, or alert on it	`order.tier = "gold"`, `payment.provider = "stripe"`
Nested array / map	The structure *is* the payload — you carry it for context and read it on a single span	A validation-error list, a resolved routing table, a request body echo

Reach for nesting only in the second case. If you find yourself planning to aggregate on something inside a nested attribute, promote that value to its own flat scalar attribute alongside the structure.

7. OTLP Metric Dimensions

Advanced OTLP Metric Dimensions

An opt-in setting changes which attributes become metric dimensions. With **Advanced OTLP metric dimensions** on, *"Dynatrace ingests all resource, scope, and data-point attributes as metric dimensions by default, except for attributes on the Deny list: all attributes."* The same page notes that *"dimension keys that were previously normalized*

are now ingested as-is", and that *"Relaxed ingestion limits apply when this feature is enabled."*

It also changes how explicit-bucket histograms are stored: with the setting on they map to a Dynatrace **Histogram**; with it off, to a **Counter**.

Before enabling it, list the dimensions your SLOs, alerts and dashboards filter on (including any `dt.entity.service` filters on OTLP metrics). Afterwards, check that each still exists with the same key and case.

Scope Attributes as Dimensions

With Advanced OTLP metric dimensions on, *"Meter name and version are always added as dimensions (otel.scope.name and otel.scope.version) for Grail-ingested metrics, regardless of the Add Meter name and version as metric dimensions setting."* With it off, the **Add Meter name and version as metric dimensions** setting controls them.

> **Sources:** Configure OTLP metrics ingestion (DT docs), About OTLP metrics ingest (DT docs) — instrument mapping table.

OTLP Auto-Configuration for Kubernetes

Dynatrace Operator (v1.8+) supports **automatic OTLP exporter configuration** for K8s workloads:

Feature	Details
Auto-injection	Environment variables injected into application pods at startup
Routing	Traffic routes through in-cluster ActiveGate if available
Control	Per-pod opt-in/out via `otlp-exporter-configuration.dynatrace.com/inject` annotation
Docs	OTLP Auto-Config

For more details, see Configure OTLP Metrics Ingestion.

```
// Verify OTel traces are arriving over OTLP
// dt.openpipeline.source identifies the ingest path; otel.scope.name also
appears on OneAgent spans.
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| summarize count = count(), by:{service.name, otel.scope.name}
| sort count desc
| limit 20
```

```
// Check recent OTel spans (OTLP-ingested)
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| fields start_time, service.name, span.name, duration
| sort start_time desc
| limit 30
```

```
// View OTel data by instrumentation scope (OTLP-ingested spans)
fetch spans, from:-1h
| filter dt.openpipeline.source == "/api/v2/otlp/v1/traces"
| summarize count = count(), by:{otel.scope.name}
| sort count desc
| limit 10
```

8. Verification

Verify Data in Dynatrace

1. **Services**: Go to **Services** - OTel services appear with OTel icon
2. **Traces**: Go to **Distributed Traces** - Filter by service
3. **Metrics**: Go to **Metrics** - Search for custom metric names
4. **Logs**: Go to **Logs & Events** - Filter by trace_id

Troubleshooting Checklist

Check	How
Token scopes	Settings > Access tokens
Collector logs	`otelcol-contrib --config config.yaml`

Check	How
Network connectivity	`curl -v https://{env}.live.dynatrace.com/api/v2/otlp`
service.name set	Check resource attributes

Common Issues

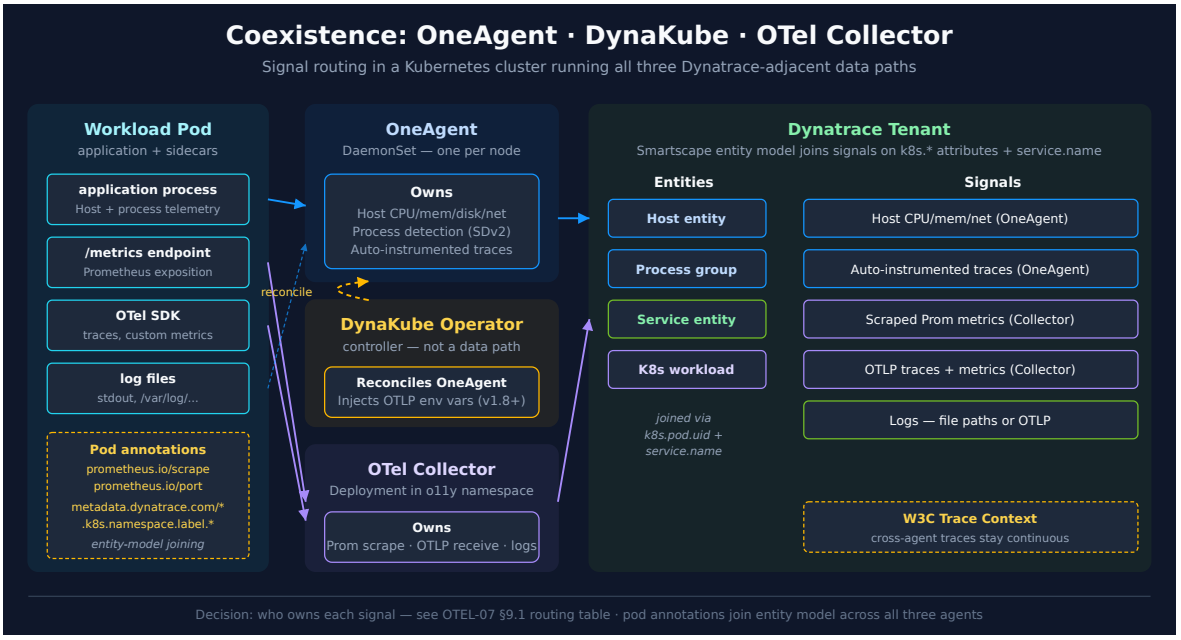
Issue	Cause	Fix
401 Unauthorized	Invalid token	Check token, regenerate
403 Forbidden	Missing scope	Add required scopes
Service shows as `unknown_service`	service.name missing	Add the resource attribute
Request rejected over payload format	JSON payload — *"Binary format must be used"*	Send binary protobuf (`OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf`)
Partial data	Network timeout	Check batch size, retry config

9. Coexistence with OneAgent and DynaKube

In Kubernetes you may have three Dynatrace-adjacent data paths active at once:

- **OneAgent** — host-process auto-instrumentation, deployed by the DynaKube operator.
- **OTel Collector** — your own deployment, scraping Prometheus targets or accepting OTLP from apps.
- **OTLP auto-config** — DynaKube Operator (v1.8+) can inject OTLP exporter env vars into application pods.

These do not conflict on the wire, but they overlap on what telemetry they produce. The decision is **which agent handles which signal**.



9.1. Signal-Routing Decision Table

In community practice, placement tends to follow the table below — confirm it against your own OneAgent coverage before committing to it:

Signal type	Preferred path	Why
Host CPU / memory / disk / network	OneAgent (DynaKube)	Auto-discovers; tightly integrated with the Smartscape host entity
Process-level metrics, process group detection	OneAgent where installed; otherwise Collector `hostmetrics` + the OpenTelemetry Host Monitoring extension	OneAgent detects process groups automatically; the extension generates host and process entities from Collector data
Application traces (OneAgent-supported language)	OneAgent	Auto-instrumentation, no code change
Application traces (unsupported language or fine-grained custom)	OTel SDK → Collector → OTLP	OTel covers languages OneAgent does not

Signal type	Preferred path	Why
Prometheus-exposed metrics from third-party / business apps	OTel Collector `prometheus` receiver → OTLP	Scales horizontally with a Target Allocator; a Dynatrace Prometheus extension (run on a OneAgent host or remotely on an ActiveGate) is the alternative when you want host-context enrichment
Custom application metrics	OTel SDK → Collector → OTLP	Same as Prometheus path
Logs from OneAgent-instrumented hosts	OneAgent	Auto-detection from log paths
Logs from OTel-only apps	OTel SDK → Collector → OTLP	OneAgent does not pick them up without the agent installed

9.2. Entity Model Joining via `metadata.dynatrace.com/*`

When the OTel Collector scrapes a pod, Dynatrace does not automatically know how to link the resulting metrics to existing entities (the service detected by OneAgent, the K8s deployment in DynaKube's view). The `k8sattributes` processor plus a pod-annotation convention solves this:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  template:
    metadata:
      annotations:
        # Standard Prometheus scrape contract (see OTEL-05 §6.2)
        prometheus.io/scrape: "true"
        prometheus.io/port: "8000"

        # Dynatrace entity-metadata hints — picked up by k8sattributes
processor
        metadata.dynatrace.com/k8s.namespace.label.domain: sandbox
        metadata.dynatrace.com/k8s.namespace.label.otel.service.name: myapp
```

The collector's `k8sattributes` processor pulls these annotations into the resource attributes of every scraped metric. A downstream `transform` processor can then merge the JSON blob into top-level attributes. Result: scraped metrics carry `service.name = "myapp"` matching what OneAgent or the OTel SDK produced for the same workload — and Dynatrace's entity model joins them.

9.3. Counter Temporality — `cumulativetodelta`

Prometheus emits **cumulative** counters that only ever increase. Dynatrace requires **delta** temporality — the per-interval change: *"The Dynatrace backend exclusively works with delta values and requires the respective aggregation temporality."* Its instrument mapping gives a cumulative Counter no Dynatrace metric type, so Prometheus counters must be converted before export. The collector's `cumulativetodelta` processor performs the conversion in-pipeline:

```
processors:
  cumulativetodelta:
    max_staleness: 3m
```

For full pipeline shape including this processor see **OTEL-05 §6.5 Full Pipeline**.

9.4. Auth Scheme Reminder

OTLP from a Collector uses `Authorization: Api-Token ${TOKEN}` (classic) or `Authorization: Bearer ${TOKEN}` (Platform Token). DynaKube's data-ingest token follows the same rule — match the scheme to the token prefix. See **§2 Authentication Setup → Token Type and Auth Scheme** for the matrix.

9.5. Trace Context Propagation

OneAgent and OTel both support W3C Trace Context, so a request that crosses agent boundaries produces a single trace in Dynatrace:

```
Service A (OneAgent) → Service B (OTel) → Service C (OneAgent)
                ↓                  ↓
            Dynatrace traces show complete path
```

Configure OTel to use W3C propagation explicitly — some SDKs default to other formats:

```
from opentelemetry.propagate import set_global_textmap
from opentelemetry.propagators.composite import CompositePropagator
from opentelemetry.propagators.w3c.traceparent import W3CTraceparentPropagator
from opentelemetry.propagators.w3c.tracestate import W3CTracestatePropagator

set_global_textmap(CompositePropagator([
    W3CTraceparentPropagator(),
    W3CTracestatePropagator()
]))
```

9.6. Managing Collector Fleets with Bindplane

Once a team runs more than a handful of OTel Collectors, the work shifts from writing one configuration to keeping many of them consistent. Dynatrace documents **Bindplane** for that job: *"a unified telemetry pipeline to manage OpenTelemetry Collector fleets centrally"*. It manages collectors remotely over OpAMP, the open protocol for collector management, and it fits the open-foundation model this series follows. The same fleet can *"send data to multiple backends simultaneously, including Dynatrace and Splunk."* Dynatrace announced an agreement to acquire Bindplane on 04/08/2026.

Question	Where Bindplane fits
Many Collectors drifting apart in configuration	Central fleet management from one control plane
Telemetry that must reach Dynatrace **and** another backend	Multi-destination routing from the same Collectors
Deployment model	Cloud or self-hosted

Two practical notes:

- **Bindplane does not change the coexistence decisions above.** It manages *Collectors*; the signal-routing table in §9.1 and the Span Sensor vs OTLP rule still decide which component emits which signal.
- **Do not write into `dt.bindplane.*`.** From SaaS 1.346 that namespace is *"reserved exclusively for Bindplane"*. A processing rule that sets those fields should move to a namespace you own.
- **Bindplane is optional.** It helps operate many Collectors; it is not a prerequisite for sending OTel data to Dynatrace. As with any SaaS sprint, rollout to tenants is staged — check the tenant version.

> **Sources:**
>
> - k8sattributes processor (OpenTelemetry Collector Contrib GitHub)
> - Cumulative to Delta processor (OpenTelemetry Collector Contrib GitHub)
> - Configure OTLP Metrics (DT docs)
> - Dynatrace Operator (Dynatrace GitHub)
> - About OTLP metrics ingest (DT docs) — *"The Dynatrace backend exclusively works with delta values and requires the respective aggregation temporality."*
> - Bindplane (DT docs) — *"Bindplane is a unified telemetry pipeline to manage OpenTelemetry Collector fleets centrally"*; *"Collectors can send data to multiple backends simultaneously, including Dynatrace and Splunk."*
> - Dynatrace to acquire Bindplane (Dynatrace press release, 04/08/2026) — *"today announced it has signed a definitive agreement to acquire Bindplane"*
> - What's new in Dynatrace SaaS 1.346 (DT docs) — *"The `dt.bindplane.*` namespace is now reserved exclusively for Bindplane."*
> - OpenTelemetry Host Monitoring extension (DT docs) — *"This extension automatically generates entities for hosts and their running processes."*
> - Prometheus data source (DT docs) — *"For high-volume Prometheus scraping in Kubernetes, and for new deployments, consider the OpenTelemetry Collector"*

Summary

In this notebook, you learned:

- Dynatrace OTLP endpoints (HTTP only — gRPC requires Collector)
- Dynatrace OTel Collector distribution for production deployments
- API token creation with required scopes; classic `Api-Token` vs Platform `Bearer` auth schemes
- Complete Collector configuration
- Direct SDK export configuration
- Entity mapping and resource attributes
- Span attributes that enhance Dynatrace analysis
- OTLP metric dimensions changes and auto-configuration

- Verification and troubleshooting
- Coexistence patterns across OneAgent, DynaKube, and the OTel Collector — signal routing, entity-model joining via `metadata.dynatrace.com/*`, counter temporality
- Managing OTel Collector fleets centrally with Bindplane, and the reserved `dt.bindplane.*` namespace

---

References

- Dynatrace OpenTelemetry
- OTLP API Endpoints
- Dynatrace Collector
- Configure OTLP Metrics
- OTLP Auto-Config for K8s
- Ensure Success with OTel
- Bindplane

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.